Internal Security Audit Report

1. Executive summary

This internal audit reviews the GameFi Economy protocol. The system includes ERC-20 governance and resource tokens, ERC-1155 items, an upgradeable parameter module, crafting, loot drops, an ERC-1155 rental vault, a constant-product AMM, an ERC-4626 treasury vault, a Chainlink price feed adapter, OpenZeppelin Governor and Timelock governance, a frontend, and a subgraph.

The review focused on access control, reentrancy, oracle safety, governance safety, ERC-4626 accounting, AMM invariants, upgradeability, and the specific failure cases required by the course. The repository includes before-and-after vulnerability case studies for reentrancy and missing access control. Production contracts use AccessControl for privileged functions, ReentrancyGuard or Checks-Effects-Interactions for external transfer flows, SafeERC20 for ERC-20 interactions, and pull payments for rental income.

At final submission time, the team must attach the actual Slither output and confirm zero High and zero Medium findings. This generated report includes the expected structure and current manual findings.

2. Scope

Commit hash

Replace before submission:

```
COMMIT_HASH=<final git commit hash>
```

Files in scope

- `contracts/GameToken.sol`
- `contracts/ResourceToken.sol`
- `contracts/GameItems1155.sol`
- `contracts/GameParametersV1.sol`
- `contracts/GameParametersV2.sol`
- `contracts/CraftingManager.sol`
- `contracts/LootDrop.sol`
- `contracts/RentalVault.sol`
- `contracts/AMMPool.sol`
- `contracts/AMMPoolFactory.sol`
- `contracts/GameVault4626.sol`
- `contracts/PriceFeedAdapter.sol`
- `contracts/GameGovernor.sol`
- `contracts/math/AMMMath.sol`
- `script/Deploy.s.sol`
- `script/VerifyPostDeploy.s.sol`

Files out of scope

• `test/helpers/VulnerabilityTargets.sol`, except as educational case studies.

• `contracts/mocks/*`, except as test support.

• Frontend and subgraph, except for high-level integration assumptions.

3. Methodology

The audit methodology combined automated and manual review:

• 1. Run `forge fmt --check` to enforce formatting.

• 2. Run `forge build` to confirm compiler success.

• 3. Run `forge test -vvv` to execute unit, fuzz, invariant, fork, and vulnerability case-study tests.

• 4. Run `forge coverage --report summary --report lcov` to measure line coverage.

• 5. Run Slither with high and medium severity failures enabled.

6. Manually inspect all privileged functions for role checks.

7. Manually inspect ETH and token transfers for CEI, ReentrancyGuard, SafeERC20, and checked return values.

8. Manually inspect oracle usage for stale-data checks and invalid-answer handling.

9. Manually inspect governance settings and Timelock role handoff.

10. Review upgradeable storage layout for collisions.

4. Findings table

ID	Severity	Title	Status
H-01	High	Reentrancy in educational vulnerable bank	Fixed in case study FixedRewardBank
H-02	High	Missing access control in educational vulnerable minter	Fixed in case study GuardedMinter
M-01	Medium	Deployer admin backdoor if roles are not revoked	Fixed in deployment script handoff
M-02	Medium	Stale oracle prices could be accepted without updatedAt checks	Fixed in PriceFeedAdapter
L-01	Low	AMM does not support fee-on-transfer tokens	Acknowledged by design
L-02	Low	Open Timelock executor allows anyone to pay execution gas	Acknowledged; improves liveness
I-01	Informational	Fork tests skip when MAINNET_RPC_URL is absent	Acknowledged; CI should set secret
G-01	Gas	Yul quote function reduces arithmetic overhead	Fixed/implemented

5. Finding details

H-01: Reentrancy in educational vulnerable bank

Severity: High

Location: `test/helpers/VulnerabilityTargets.sol:VulnerableRewardBank.withdraw`

Description: The vulnerable case-study contract sends ETH to the caller before setting the caller's balance to zero. A malicious receiver can reenter and withdraw multiple times.

Impact: If this pattern existed in production, an attacker could drain ETH from the contract.

Proof of concept: `testCaseStudyReentrancyBeforeExploitDrainsVulnerableBank` demonstrates the exploit.

Recommendation: Update internal accounting before external calls and add ReentrancyGuard.

Status: Fixed in `FixedRewardBank`. Production rental withdrawals use pull payments and set pending balances to zero before calling the receiver.

H-02: Missing access control in educational vulnerable minter

Severity: High

Location: `test/helpers/VulnerabilityTargets.sol:UnguardedMinter.mint`

Description: The vulnerable case-study minter allows any caller to mint arbitrary balances.

Impact: If this pattern existed in production, attackers could inflate supply and break the economy.

Proof of concept: `testCaseStudyAccessControlBeforeAnyoneCanMint` demonstrates unauthorized minting.

Recommendation: Protect minting functions with AccessControl or Ownable.

Status: Fixed in `GuardedMinter`. Production minting functions use AccessControl roles.

M-01: Deployer admin backdoor if roles are not revoked

Severity: Medium

Location: `script/Deploy.s.sol`

Description: A deployer that retains DEFAULT_ADMIN_ROLE or mint roles after deployment could bypass the DAO.

Impact: A compromised deployer key could update parameters, mint tokens, mint items, pause contracts, or release treasury assets.

Proof of concept: Manual review of AccessControl role ownership after deployment.

Recommendation: Grant Timelock all required roles, grant modules their minimum roles, then revoke deployer admin roles.

Status: Fixed. The deployment script grants roles to Timelock and modules and revokes deployer privileged roles.

M-02: Stale oracle prices could be accepted without updatedAt checks

Severity: Medium

Location: `contracts/PriceFeedAdapter.sol`

Description: Oracle integrations that only read the answer can accept stale or incomplete rounds.

Impact: Game economy functions could be based on stale prices, causing incorrect treasury or marketplace behavior.

Proof of concept: `testPriceFeedStaleReverts` verifies stale price rejection.

Recommendation: Require positive answer, nonzero timestamp, and `block.timestamp - updatedAt <= maxStaleness`.

Status: Fixed.

L-01: AMM does not support fee-on-transfer tokens

Severity: Low

Location: `contracts/AMMPool.sol`

Description: The AMM assumes the transferred amount equals `amountIn`. Fee-on-transfer tokens would make reserves differ from expected values.

Impact: The pool could quote inaccurate outputs for fee-on-transfer tokens.

Recommendation: Only use approved standard resource tokens, or modify swap logic to calculate actual amount received.

Status: Acknowledged. The factory is intended for project-owned `ResourceToken` assets only.

L-02: Open Timelock executor

Severity: Low

Location: `script/Deploy.s.sol`

Description: The Timelock executor role is granted to address zero, allowing anyone to execute queued operations after the delay.

Impact: Anyone can pay gas to execute a queued proposal. This is not a bypass because only queued proposals can be executed.

Recommendation: Keep open executor for liveness or restrict to trusted keepers if project policy changes.

Status: Acknowledged.

I-01: Fork tests skip without RPC URL

Severity: Informational

Location: `test/GameFiForkTest.t.sol`

Description: Fork tests return early if `MAINNET_RPC_URL` is not configured.

Impact: Local developers without RPC access can still run the suite. CI must set the secret to perform real fork interactions.

Recommendation: Add `MAINNET_RPC_URL` to CI secrets before final run.

Status: Acknowledged.

G-01: Yul quote function

Severity: Gas

Location: `contracts/math/AMMMath.sol`

Description: `quoteOutYul` computes the AMM quote using inline assembly. It is benchmarked against `quoteOutSolidity`.

Impact: Slight gas savings are expected on hot AMM quote paths.

Recommendation: Keep both functions and document benchmark results in `docs/GAS_REPORT.md`.

Status: Implemented.

6. Reentrancy analysis

External-call paths were reviewed:

- `AMMPool.swap`, `addLiquidity`, and `removeLiquidity` use ReentrancyGuard and SafeERC20.
- `RentalVault.list`, `rent`, `withdrawListing`, and `claimEarnings` use ReentrancyGuard where state and external calls interact.
- `RentalVault.claimEarnings` follows CEI by setting pending balance to zero before ETH transfer.
- `GameVault4626` deposit, mint, withdraw, redeem, and treasury release use ReentrancyGuard.
- `CraftingManager.craft` and `LootDrop.openLootBox` use ReentrancyGuard.

The educational vulnerable reentrancy target intentionally violates CEI. It is not in production scope.

7. Access-control analysis

All privileged production functions use AccessControl:

- Token minting and burning are role-gated.
- Item minting, burning, URI changes, and pausing are role-gated.
- Crafting and loot configuration are role-gated.
- Recipe and loot table updates are role-gated and intended for Timelock.
- UUPS upgrade authorization is role-gated through `UPGRADER_ROLE`.
- Treasury releases are role-gated through `TREASURY_ROLE`.
- Rental fee updates are role-gated.
- Pool creation and pool pausing are role-gated through the factory.

No production authorization path uses `tx.origin`.

8. Oracle attack analysis

Stale price

Attack: Use old data from a Chainlink feed.

Defense: `PriceFeedAdapter.latestPrice` checks `updatedAt != 0` and requires the answer to be within `maxStaleness`.

Negative or zero price

Attack: Consume an invalid answer.

Defense: `answer > 0` is required.

Feed depeg or incorrect feed selection

Attack: Use the wrong feed address or a depegged feed.

Defense: Deployment is parameterized and documented. The team must verify feed address and decimals before final deployment. Governance can update protocol parameters if a feed issue affects economic assumptions.

Randomness manipulation

Attack: Influence loot drops with block timestamp, block hash, or miner-controlled values.

Defense: `LootDrop` uses a VRF coordinator interface and stores request ids. It does not use block timestamp, block hash, or block number as randomness.

9. Governance attack analysis

Flash-loan governance attacks

Attack: Borrow voting tokens, vote, then return them.

Defense: OpenZeppelin ERC20Votes uses checkpoints. Voting power is measured at the proposal snapshot block, not at the current block. A same-block flash-loan cannot create votes at the snapshot retroactively.

Whale attacks

Attack: A large holder passes malicious proposals.

Defense: This cannot be eliminated in token voting. Mitigations are quorum, proposal threshold, public Timelock delay, monitoring, and token distribution discipline.

Proposal spam

Attack: Create many proposals to overwhelm voters.

Defense: Proposal threshold is 1% of initial supply, making spam expensive.

Timelock bypass

Attack: Directly call sensitive contracts without waiting.

Defense: Deployment script hands roles to Timelock and revokes deployer roles. Sensitive functions require roles controlled by Timelock.

Malicious upgrade

Attack: Upgrade `GameParameters` to malicious implementation.

Defense: Upgrade requires `UPGRADER_ROLE`, which is assigned to Timelock after deployment. The 2-day delay allows review before execution.

10. Centralization analysis

The deployer is powerful during deployment only. The script revokes deployer privileged roles after Timelock setup. Before final submission, the team must run the post-deployment script and verify no deployer backdoor remains.

The DAO can update parameters, upgrade the parameter proxy, release treasury funds, and pause certain contracts. This is intentional centralization through governance. Users trust token governance and the Timelock delay.

Chainlink is an external dependency. If Chainlink feed data or VRF service fails, affected functions may revert or wait. This is preferable to accepting unsafe data.

The frontend and subgraph are not trusted for correctness. Users can interact directly with contracts. The frontend displays subgraph data but sends transactions to contracts.

11. AMM economic review

The AMM uses the standard fee-adjusted constant product formula:

```
amountInWithFee = amountIn * 997
amountOut = reserveOut * amountInWithFee / (reserveIn * 1000 + amountInWithFee)
```

This corresponds to a 0.3% input fee. Swap calls require `minAmountOut`, protecting traders from unexpected price movement. The pool updates reserves after transfers and asserts that k does not decrease after a swap.

Known limitation: the pool is designed for standard ERC-20 resource tokens only, not rebasing or fee-on-transfer assets.

12. ERC-4626 review

`GameVault4626` inherits OpenZeppelin ERC4626 and wraps deposit, mint, withdraw, and redeem with ReentrancyGuard and Pausable. Tests include deposit, mint, withdraw, redeem, and fuzz coverage for rounding behavior.

The treasury release function can transfer underlying assets from the vault. This is intentional because Timelock controls treasury spending. Governance proposals should describe any treasury release clearly.

13. Upgradeability review

Only `GameParameters` is upgradeable. This minimizes upgrade risk. V1 storage includes recipes, loot table arrays, and a storage gap. V2 adds functions but no storage variables. Therefore, V2 cannot collide with V1 state.

Before any future upgrade, the team must:

- 1. Add a storage layout diff to the architecture document.
- 2. Write an upgrade test.
- 3. Queue the upgrade through Timelock.
- 4. Wait 2 days.
- 5. Execute and verify version/state preservation.

14. Slither appendix

Run before final submission:

```
slither . --filter-paths "test|script|lib" --exclude-dependencies --fail-medium
```

Paste the output here:

```
<attach final Slither output here>
```

Expected final condition: zero High and zero Medium findings. Low and informational findings must be listed in the findings table above with explicit justification.

15. Final audit checklist

- [ ] No `tx.origin` usage.
- [ ] No `transfer` or `send` usage for ETH.
- [ ] All ERC-20 interactions use SafeERC20 or controlled mint/burn interfaces.
- [ ] All external value-transfer paths use CEI or ReentrancyGuard.
- [ ] Every privileged function is role-gated.
- [ ] Timelock controls treasury and parameters.
- [ ] Deployer roles are revoked after deployment.
- [ ] Price feed rejects stale and invalid data.
- [ ] VRF randomness does not use block values.
- [ ] UUPS upgrade path is tested.
- [ ] AMM k invariant is tested.
- [ ] ERC-4626 rounding behavior is fuzzed.